Detecting Process Concept Drifts from Event Logs

Canbin Zheng, Lijie Wen(✉), and Jianmin Wang

School of Software, Tsinghua University, Beijing 100084, China
zcb0821@outlook.com, {wenlj,jimwang}@tsinghua.edu.cn

Abstract. Traditional process discovery algorithms assume processes to be in a steady state. However, process models tend to be dynamic due to various factors, which has brought challenges such as change point detection, change localization and change process discovery. Existing techniques to identify change points are sensitive to parameters and the accuracy is not satisfactory. This paper proposes a novel approach to deal with such concept drift phenomenon. Event logs can be characterized by the relationships between activities, which motivates us to transform a log into a relation matrix. By detecting the always and never intervals in each row of the relation matrix, we obtain candidate change points for each relation. Finally, all the candidate change points are combined into an overall result. The approach is also able to localize the changes between different phases. Experiments on synthetic logs show that our approach is accurate and performs better than the state of the art in detecting sudden drift.

Keywords: Process mining · Concept drift · Process discovery · Change detection

1 Introduction

A business process refers to logically related activities or tasks conducted by people or equipments to accomplish a predetermined goal. Executions of processes are recorded in event logs including information about which activities are performed, at what time, by whom, in the context of which case, etc. Process mining techniques aim to extract knowledge from event logs commonly available in today's information systems [1]. The most crucial learning task in the process mining domain is termed process discovery, which is capable of constructing a process model (i.e. Petri nets) that accurately describes the process, as it takes place in real life. A majority of process discovery techniques like α-algorithm [18], Heuristic Miner [23], Genetic Miner [20], ILP Miner [9], etc. assume processes to be in a steady state, i.e. one event log corresponds to just one process model.

However, process models tend to be dynamic due to various factors like structure optimization, demand and supply, emergency, etc. Some changes are planned and documented, but some occur unexpectedly and may remain unnoticed. Bose et al. analyzed processes in more than 100 organizations and found

© Springer International Publishing AG 2017
H. Panetto et al. (Eds.): OTM 2017 Conferences, Part I, LNCS 10573, pp. 524–542, 2017.
https://doi.org/10.1007/978-3-319-69462-7_33

that it is very unrealistic to assume that the process being studied is in a steady state [4]. In predictive analysis and machine learning, "concept drift" means that the statistical properties of the target variable, which the model is trying to predict, change over time in unforeseen ways. Here we use concept drifts to refer to situations where the process is changing while being analyzed. An event log may correspond to multiple models serving in succession.

If we apply traditional process discovery methods to an event log with concept drifts, we will obtain a composite process model which fails to reflect the staged variation. Besides, a composite process model may be large and complex. The evolution of process models has brought challenges to process mining, such as change point detection, change localization and change process discovery. Change point detection aims to identify the time periods when changes have taken place, which is the first and most fundamental problem because change localization and change process discovery are both based on the drift detection result.

Offline analysis and online analysis are two major classes in dealing with concept drifts. Offline analysis refers to scenarios where analysts are given the entire log data from the beginning. In contrast, online analysis is one that can only deal with its input piece-by-piece in a serial fashion, i.e., in the order that the traces or events are fed to the algorithm, without having the entire log from the start.

Facing concept drifts, process analysts require methods and tools that are capable of detecting change points from event logs, helping them to analyze the evolution of processes. Literatures [2,4,5,12,13] have provided techniques to deal with concept drift problem. Some base on abstract interpretation of traces and some base on distance of activities. The mainstream method borrows ideas from *change-point detection* of which the goal is to detect changes in the generating distributions of the time-series. It performs statistical hypothesis testing on the data stream transformed from trace stream by extracting features. The performance of these techniques is not satisfactory. Parameters like window size have critical influences on the accuracy and are difficult to tune. Besides, there is a lack of theoretical illustration in their ability to recognize changes.

In this study, we focus on offline concept drift scenarios and describe a three-stage approach called TPCDD (Tsinghua Process Concept Drift Detection). First, each trace is represented by multiple relations. And then for each relation, we inspect its variation trend and partition it. Finally, we combine all change points revealed by each relation to get the final result. TPCDD achieves high accuracy and the parameter setting can fit in different scale of logs. Besides, its recognition ability is extendable by customizing relations and adding them into the relation matrix. In the meantime of identifying change points, TPCDD is also able to localize the changes.

The rest of this paper is organized as follows. Section 2 introduces basic concepts used throughout the paper. Our approach for concept drift detection (i.e., TPCDD) is described in Sect. 3, including three major steps. Section 4 provides an evaluation of TPCDD on synthetic logs and compare it with the

state of the art. Section 5 discusses related work. Section 6 concludes our work and points out our future directions.

2 Preliminaries

This section introduces some basic concepts used throughout the paper. Petri nets has been proved to be a powerful modeling language for business processes [19]. We do our research on a subset of Petri nets called WF-net.

Definition 1 (Petri nets). *Let $P, T \subseteq U$ be finite and disjoint sets such that $P \cup T \neq \emptyset$ and $P \cap T = \emptyset$, $F \subseteq (P \times T) \cup (T \times P)$. The tuple $N = (P, T, F, M_0)$ is a net system, where P is the set of places, T is the set of transitions and F is the set of arcs. A marking M of N assigns each place a natural number $M(p)$ of tokens. M_0 is the initial marking of the net.*

We write $\bullet y = \{x|(x, y) \in F\}$ and $y\bullet = \{x|(y, x) \in F\}$ for the pre-set and post-set of a node y respectively. The semantics of a net system N are typically given by a set of sequential runs. A transition t of N is enabled at a marking M_i of N iff $\forall p \in \bullet t, M_i(p) \geq 1$. If t is enabled at M_i, then t can occur and afterwards N reaches a new marking M_{i+1} where $\forall p \in P$, $M_{i+1}(p) = M_i(p) - 1$ if $p \in (\bullet t \setminus t\bullet)$, $M_{i+1}(p) = M_i(p) + 1$ if $p \in (t\bullet \setminus \bullet t)$, and $M_{i+1}(p) = M_i(p)$ otherwise. An occurrence sequence of N is a transition sequence (e.g., $t_1 t_2 t_3 \ldots$) in which transitions t_1, t_2, t_3, *etc.* occur in succession.

Definition 2 (Workflow net or WF-net). *A Petri net system $N = (P, T, F, M_0)$ that models the control-flow dimension of a workflow is called a workflow net (WF-net for short), which should satisfy the following requirements:*

- *There is one unique source place i such that $\bullet i = \emptyset$;*
- *There is one unique sink place o such that $o\bullet = \emptyset$;*
- *Every node $x \in P \cup T$ is on a path from i to o.*

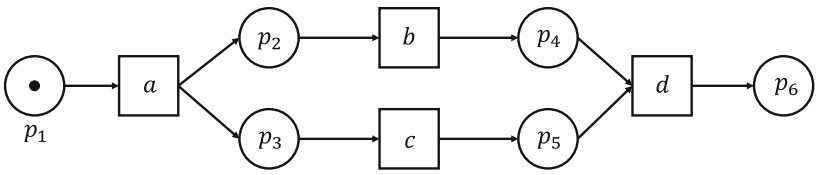


Fig. 1. A simple example of WF-net

Figure 1 shows an example of WF-net, in which $P = \{p_1, p_2, p_3, p_4, p_5, p_6\}$, $T = \{a, b, c, d\}$. There is only one token in p_1 representing the initial marking M_0. Taking a as example, $\bullet a = \{p_1\}$ and $a\bullet = \{p_2, p_3\}$. a is enabled at M_0. Once a occurs, the token in place p_1 will be consumed and the numbers of tokens in

p_2 and p_3 will be increased by 1 respectively. For the WF-net in Fig. 1, there are two possible occurrence sequences *abcd* and *acbd*.

Process mining aims to discover, monitor and improve real processes by extracting knowledge from event logs available in today's information systems. The starting point for process discovery is an event log. Each event in such a log refers to an activity (i.e., a well-defined step in a process) and is related to a particular case (i.e., a process instance). The events belonging to a case are ordered and can be seen as one run of the process. Event logs may store additional event attributes. In fact, whenever possible, process mining techniques use attributes such as the resource (i.e., person or device) executing or initiating the activity, the timestamp of the event, or data elements recorded with the event. In this paper, we abstract each process instance as a trace and each event log as a multiset of traces.

Definition 3 (Trace, Event log). *Let P_M be a process model and A be a set of activities. A^+ is the set of all nonempty finite sequence of activities from A. $\sigma \in A^+$ is called a trace when σ represents a firing activity sequence of PM. A log is a multiset of traces.*

Occurrence sequences of a net system N correspond to traces in the event log. For example, there are two possible traces *abcd* and *acbd* for the model in Fig. 1. The multiset $[abcd, abcd, abcd, acbd, acbd]$ is an event log with 5 cases, in which trace *abcd* occurred 3 times while *acbd* occurred 2 times.

We present some basic notations used in concept drift detection.

Definition 4 (Concept drift). *Let $PM_0, PM_1, \cdots, PM_n$ be $n+1$ different process models and $T_0 < T_1 < \cdots < T_n$ be $n+1$ moments. $PM(T_i) = PM_i$ denotes the model being used at T_i. $PM(T_0) = PM_0$ is the initial model. When moment T_i $(0 < i \leq n)$ arrives, the currently used model will change into PM_i immediately. But the traces are still recorded in the same log. Such phenomenon is called concept drift. In this context, $T_1, \cdots, T_n$ are called change points.*

Note that in this paper, we focus on control flow changes which can be classified into operations such as insertion, deletion, substitution and reordering of process fragments. Data and resource perspectives are not taken into account.

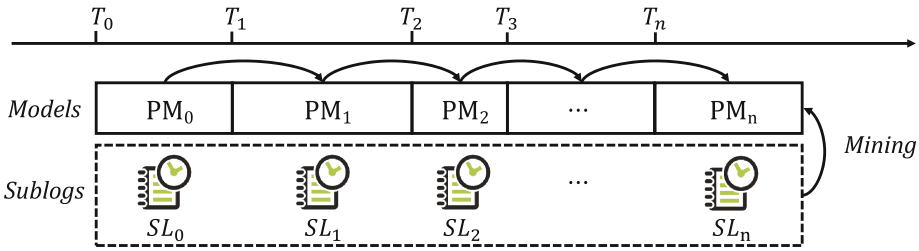


Fig. 2. Concept drift phenomenon in process mining

Besides, when model PM was changed from PM_i into PM_{i+1} at T_i, we assume that it happened in an instant (sudden drift). In other words, all traces produced at and after T_i no longer belong to model PM_i. The illustrative phenomenon for concept drifts in process mining is presented in Fig. 2.

3 Concept Drift Detection

Given a log $L = [\sigma_1, \sigma_2, \cdots, \sigma_n]$ which incorporates concept drifts, concept drift detection aims to find out the moments when the changes happened. From Definition 4, we obviously know that the model behavior is not the same before and after a change point. Take T_1 in Fig. 2 as an example. If we collect the traces between $[T_0, T_1)$ as SL_0 and the traces between $[T_1, T_2)$ as SL_1, SL_0 is different from SL_1. Therefore, a natural idea is to compare certain amount of traces before and after a candidate change point. Literatures like [4,12,13] all adopt this solution, which has two challenges. One is how to measure the differences between two sets of traces and the other is how many traces shall we collect for testing (that is to decide the window size). For the first challenge, [4,12,13] used feature extraction plus statistical hypothesis testing as a solution. For the second one, fixed and adaptive window size strategy are both introduced in [4,12,13]. However, the actual performance is not quite satisfactory because it heavily depends on the choice of window size. Once the window size is not proper, false negative and false positive are both possible to happen. To avoid such disadvantages, we proposed a totally different idea.

The topic of similarity between process models has been well studied. Existing algorithms like TAR [24], BP [22], ExRORU [21], 4C spectrum [16] offered a variety of solutions. Some can only be derived from models, some can only be derived from traces, and some can be derived from both. The principle behind

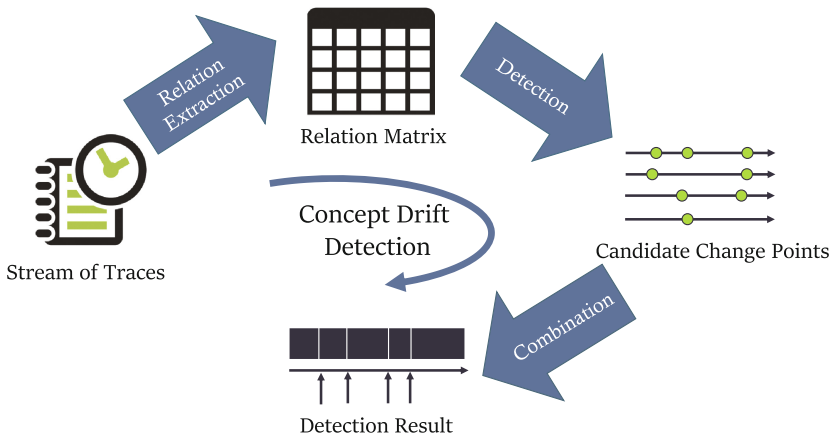


Fig. 3. Overview of our approach for detecting concept drifts

is that process models can be characterized by the relationships between activities. Motivated by similarity calculation and divide-and-conquer strategy, we proposed TPCDD to detect process concept drifts (Fig. 3).

Figure 3 provides an overview of our approach named TPCDD. First, we extract relations from each trace and transform the event log into a relation matrix. Then, for each relation, we observe its variation trend and detect candidate change points. Finally, all candidate change points are clustered to get a final result. In the following subsections, we will introduce TPCDD in detail.

3.1 Relation Extraction

Event logs can be characterized by the relationships between activities. In this section, we first introduce two basic kinds of relations and then transform a log into a relation matrix.

Definition 5 (Direct succession relation (DSR)). *Let $\sigma = t_1 t_2 \cdots t_m$ be a trace and $A = \{a_1, a_2, \cdots, a_k\}$ be a set of activities. $\forall x, y \in A$, (x, y) is a direct succession relation (denoted as $x \rightarrow y$) if and only if $\exists i \in \{1, 2, \cdots, m-1\}$ such that $t_i = x$ and $t_{i+1} = y$.*

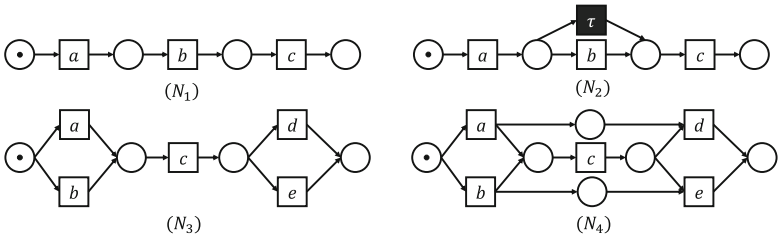
Consider an event log $L = [abcd, acbd]$. The set of DSRs of trace *abcd* is $\{a \rightarrow b, b \rightarrow c, c \rightarrow d\}$ and that of trace *acbd* is $\{a \rightarrow c, c \rightarrow b, b \rightarrow d\}$. The time complexity of extracting DSRs from a trace with k activities is $O(k)$.

Definition 6 (Weak order relation (WOR)). *Let $\sigma = t_1 t_2 \cdots t_m$ be a trace and $A = \{a_1, a_2, \cdots, a_k\}$ be a set of activities. $\forall x, y \in A$, (x, y) is a weak order relation (denoted as $x \rightsquigarrow y$) if and only if $\exists i, j \in \{1, 2, \cdots, m\}, i < j$ such that $t_i = x$ and $t_j = y$.*

Consider an event log $L = [abcd, acbd]$. The set of WORs of trace *abcd* is $\{a \rightsquigarrow b, a \rightsquigarrow c, a \rightsquigarrow d, b \rightsquigarrow c, b \rightsquigarrow d, c \rightsquigarrow d\}$ and that of trace *acbd* is $\{a \rightsquigarrow c, a \rightsquigarrow b, a \rightsquigarrow d, b \rightsquigarrow d, c \rightsquigarrow b, c \rightsquigarrow d\}$. The time complexity of extracting WORs from a trace with k activities is $O(k^2)$.

At the first glance, DSR seems redundant as $a \rightarrow b$ leads to $a \rightsquigarrow b$. In fact, DSR and WOR are both necessary. In Fig. 4, N_1 and N_2 are distinguishable in DSR but behave the same in WOR. And the case of N_3 and N_4 are in the contrary. Accordingly, DSR and WOR complement each other in model characterization.

Definition 7 (Relation matrix). *Let L be an event log with n traces, R be a list with m relations and D be a $m \times n$ matrix. $D(i, j)$ represents the value in the i-th row and j-th column of D. Then an event log can be transformed into a relation matrix D by setting $D(i, j) = 1$ if the trace $L(j)$ contains the relation $R(i)$ and $D(i, j) = 0$ otherwise.*



	N_1	N_2	N_3	N_4
DSRs	$\{(a,b),(b,c)\}$	$\{(a,b),(a,c),(b,c)\}$	$\{(a,c),(b,c),(c,d),(c,e)\}$	
WORs	$\{(a,b),(a,c),(b,c)\}$		$\left\{\begin{matrix}(a,c),(a,d),(a,e),\\(b,c),(b,d),(b,e),\\(c,d),(c,e)\end{matrix}\right\}$	$\left\{\begin{matrix}(a,c),(a,d)\\(b,c),(b,e),\\(c,d),(c,e)\end{matrix}\right\}$

Fig. 4. DSR and WOR comparison for necessity analysis

For example, given an event log $L = [abcd, acbd, acbd, abcd, acd, abcd, acbd, ad, abd]$, Table 1 shows L's relation matrix defined over DSRs.

Table 1. The relation matrix of L defined over DSRs

	$L(1)$	$L(2)$	$L(3)$	$L(4)$	$L(5)$	$L(6)$	$L(7)$	$L(8)$	$L(9)$
$a \rightarrow b$	1	0	0	1	0	1	0	0	1
$a \rightarrow c$	0	1	1	0	1	0	1	0	0
$a \rightarrow d$	0	0	0	0	0	0	0	1	0
$b \rightarrow c$	1	0	0	1	0	1	0	0	0
$b \rightarrow d$	0	1	1	0	0	0	1	0	1
$c \rightarrow b$	0	1	1	0	0	0	1	0	0
$c \rightarrow d$	1	0	0	1	1	1	0	0	0

Let k be the average length of traces. The time complexity of converting an event log to a relation matrix is $O(nk) + O(nk^2) = O(nk^2)$.

DSR and WOR are borrowed from process similarity algorithms (i.e., TAR and BP) respectively. We use these two kinds of relations because they can be calculated from traces efficiently. Besides, they are powerful enough for change detection in general scenarios. In fact, the relation matrix is extendable. If more particular changes are required to be detected, an alternative solution is to introduce additional relations and extract them from the log.

3.2 Candidate Change Points Detection

Changes among process models can be reflected by the variation of relations between activities in the log. In this section, we observe the variation trend for each relation through its frequency level and detect candidate change points.

Definition 8 (Frequency level). *Given an integer interval* $[p..q)$ *and a specific relation* $R(i)$*, the frequency level of* $R(i)$ *over* $[p..q)$ *is*

- *always if and only if* $\forall j \in [p..q)$, $D(i,j) = 1$,
- *never if and only if* $\forall j \in [p..q)$, $D(i,j) = 0$,
- *sometimes otherwise.*

Correspondingly, $[p..q)$ *is called as an always/never/sometimes interval.*

An *always* or *never* interval implies a steady state of a certain relation and its length indicates how long the steady state lasts. If the frequency levels of a relation over two consecutive intervals are different, changes may occur at the split point. The split point is a candidate change point. In concept drift detection, we only focus on long enough (above a threshold called the minimum relation invariance distance, addr. MRID) *always* or *never* intervals because fluctuation between short intervals are regarded as normal variations inside a single model.

According to Definition 7, the variation of a relation is modelled as a row in relation matrix D. Algorithm 1 describes how to detect *always* and *never* intervals in each row of the relation matrix. The ends of these intervals form the candidate change points.

Algorithm 1. Candidate Change Points Detection

```
   Input: the i-th row of relation matrix D[i], MRID
   Output: a set of candidate change points P
1  begin
2      P ← ∅, begin ← 0, count ← 0, n ← length of D[i]
3      for j ← 1 to n do
4          if begin = 0 or D[i][j] ≠ D[i][begin] then
5              if count ≥ MRID then
6                  P ← P ⋃ {begin, j}
7              begin ← j
8              count ← 0
9          count ← count + 1
10     if count ≥ MRID then
11         P ← P ⋃ {begin}
12     P = P \ {1}
```

Algorithm 1 detects *always* and *never* intervals by iterating over $D[i]$. Variable *begin* records the index of the value being observed and *count* records the number. As long as the currently visiting value is the same as $D[i][begin]$, *count* keeps increasing (line 9). Once the currently visiting value differs from $D[i][begin]$ and $count \geq MRID$, an *always* or *never* interval is detected (line 4–6). Afterwards, we reset *begin* and *count* for continuing detection. Let m be the number

of relations and n be the number of traces. Then the time complexity of detecting all candidate change points is $O(mn)$.

Figure 5 shows an example, where $D[i] = [0,0,0,0,0,0,1,1,1,0,0,0,0,1,1,1,1,1,1,1]$ and MRID $= 5$. Algorithm 1 will divide $D[i]$ into three intervals [1..7), [7..14) and [14..21), of which the corresponding frequency levels are *never*, *sometimes* and *always*. This means that relation $R[i]$ changed over three phases, never exists, sometimes exists, always exists and the candidate change points are $\{7, 14\}$. Note that the parameter MRID only limits the minimum length of an *always* or *never* interval, which makes the algorithm adaptively fits different scales of logs.

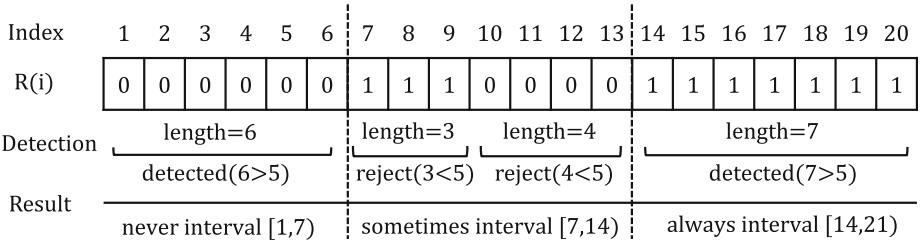


Fig. 5. An example for detecting change points of a relation

3.3 Candidate Change Points Combination

Applying Algorithm 1 to each row of a relation matrix D, we get sets of candidate change points reflecting phased variations. The procedure of combining all these candidate change points are described in Algorithm 2. Let multiset S represents all candidate change points. In concept drift context, points in S are likely to be distributed in neighborhoods of real change points, which motivates us to adopt DBSCAN[1] algorithm to partition S.

DBSCAN has two parameters: the maximum radius of a neighborhood (*eps*) and the minimum number of points required to form a dense region (*minPts*). By setting $minPts = 1$, no candidate change point will be dropped. *eps* is set to be a small value (like 10) or a small fraction (like 10%) of MRID. Line 9 ensures that the size of any partition of the log is no less than MRID, i.e. we keep the log size of each model to be at least MRID (consistent with Algorithm 1). More points in a cluster indicates more relations changed around the center of the cluster, which means that a real change point is more likely to lie in the cluster. Therefore, we check the clusters in descending order of size.

The clustering result not only contains change points, but also gives their corresponding weight (the size of the cluster). This provides convenience and freedom for the user to drop some less reliable points. For example, a cluster containing only one candidate change point can be treated as an exception rather

[1] https://en.wikipedia.org/wiki/DBSCAN.

Algorithm 2. Candidate Change Points Combination

Input: S(multiset of candidate change points), $MRID$, eps
Output: change points detected

```
begin
    minPts ← 1
    result ← [1, n]
    clusters ← DBSCAN(S, minPts, eps)
    sort clusters in descending order of size
    for i ← 1 to |clusters| do
        c ← average of clusters[i]
        find change points left and right in result such that left < c < right
        if c − left ≥ MRID and right − c ≥ MRID then
            insert c into result at the position between left and right
```

a drift. By discarding clusters which has relatively few candidate change points, we can make the difference between real drifts and exceptions.

Figure 6 shows the procedure of combining candidate change points. After applying DBSCAN method, five clusters C_1, C_2, C_3, C_4, C_5 are returned with different sizes. The size indicates how many relations are changed in a cluster. Finally, we use clustering centers of C_1, C_2, C_3, C_4, C_5 as model change points.

Let m be the number of relations, n be the number of traces, s be the MRID and k be the average length of traces. Then the number of candidate change points is no more than mn/s. Therefore, the time complexity of Algorithm 2 is $O(mn/s)$. Combining relation extraction, candidate change points detection and combination, the time complexity of the overall detection is $O(nk^2) + O(mn) + O(mn/s) = O(nk^2) + O(mn)$.

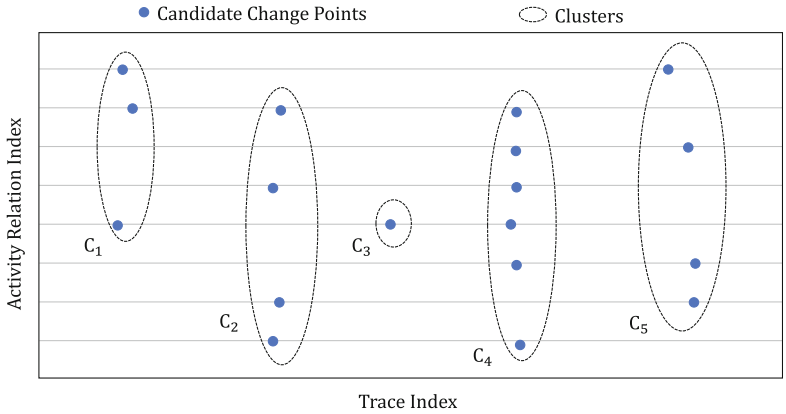


Fig. 6. The procedure of combining candidate change points

3.4 Recognition Ability

By extracting DSRs and WORs of each trace, an event log is transformed into a relation matrix and no longer used in the following steps. Therefore, we must make sure the information in the relation matrix is rich enough for distinguishing two different models. In this part, we prove that the ability of TPCDD to recognize changes is stronger than either TAR or BP.

The definition of TAR is the same as DSR. TAR algorithm distinguishes two models based on the TAR set, which contains all TARs of a model. Let S_1, S_2 be the TAR sets of model PM_1 and PM_2 respectively. If PM_1 and PM_2 are distinguishable by TAR algorithm, there exists a relation $a \rightarrow b$ such that $a \rightarrow b$ is only in one of the two TAR sets. Without loss of generality, we assume $(a \rightarrow b) \in S_1 \wedge (a \rightarrow b) \notin S_2$. Then for any log belonging to PM_2, the frequency level of $a \rightarrow b$ over any period is *never*. And for any log belonging to PM_1, the frequency level of $a \rightarrow b$ may be *always* or *sometimes*. This means that PM_1 and PM_2 are also distinguishable by TPCDD since the frequency levels of $a \rightarrow b$ are different.

BP algorithm uses the set of behavioral relations including strict order relation (denoted as SOR), exclusiveness relation (denoted as ER) and interleaving order relation (denoted as IOR) to measure the similarity of two process models. SOR, ER and IOR are all grounded on a fundamental relation called weak order relation (i.e., WOR), which is also used in our paper. If two models PM_1, PM_2 are distinguishable by BP set, then the weak order relation sets of PM_1, PM_2 are different, which means that PM_1, PM_2 are also distinguishable by TPCDD. The proof is similar to that of TAR set.

With DSRs and WORs, TPCDD possesses good capability of change recognition. Besides, it is extendable since one can customize activity relations if more particular changes are needed to be recognized.

4 Evaluations and Comparisons

4.1 Experimental Settings

To simulate the presence of concept drifts artificially, we generated mixed logs of different processes used in [12]. The base model is a textbook example of a business process for assessing loan applications (see Fig. 7), which has 15 activities, one start event and three end events, and exhibits different control-flow structures including loops, parallel and alternative branches. Maaradji et al. altered the base model by applying different types of control-flow changes, including 12 simple change patterns organized into three categories: Insertion (I), Resequentialization (R) and Optionalization (O) (shown in Table 2) and 6 composite change patterns (IOR, IRO, OIR, ORI, RIO, ROI) [12]. We modeled these processes in forms of Petri nets via PIPE[2]. The algorithm presented in this paper has been implemented[3] in Python language.

[2] https://sourceforge.net/projects/pipe2/.
[3] https://github.com/THUBPM/process-drift-detection.

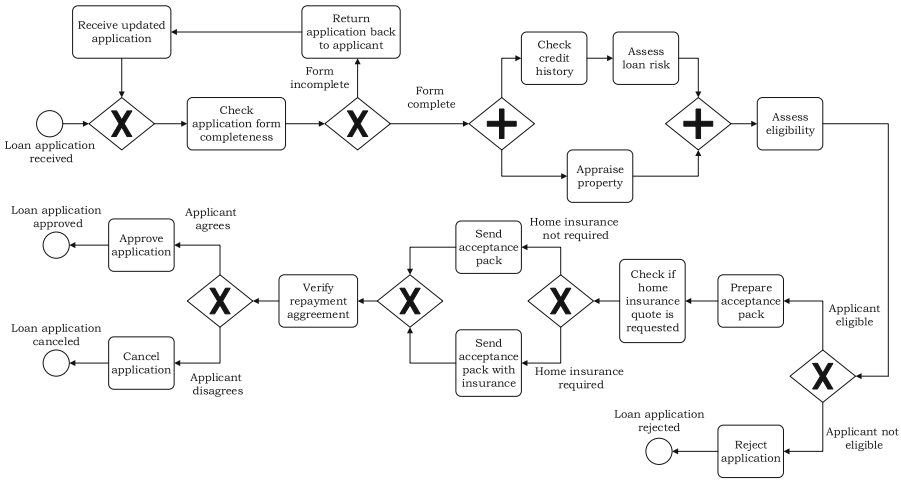


Fig. 7. A business process for assessing loan applications

Table 2. Simple control-flow change patterns

Code	Simple change pattern	Category
re	Add/remove fragment	I
cf	Make two fragments conditional/sequential	R
lp	Make fragment loopable/non-loopable	O
pl	Make two fragments parallel/sequential	R
cb	Make fragment skippable/non-skippable	O
cm	Move fragment into/out of conditional branch	I
cd	Synchronize two fragments	R
cp	Duplicate fragment	I
pm	Move fragment into/out of parallel branch	I
rp	Substitute fragment	I
sw	Swap two fragments	I
fr	Change branching frequency	O

A mixed log is determined by two parameters, the number of models (denoted as N) and the log sizes of each model (denoted as S). Given N and S, we first randomly select N adjacent different models and generate $S[i]$ traces for each model M_i ($i \in [1..N]$) and then merge these logs into one. To simulate different situations, we generated 32 mixed logs with different number of models and different sizes of logs for each model as shown in Table 3.

In Table 3, the first column indicates the log size for each model and the first row indicates the number of models in a mixed log. For example, $L_{4,500}$ is a

Table 3. Testing logs used in the experiments

	2	4	6	8	10	12	14	16
250 ± 50	$L_{2,250}$	$L_{4,250}$	$L_{6,250}$	$L_{8,250}$	$L_{10,250}$	$L_{12,250}$	$L_{14,250}$	$L_{16,250}$
500 ± 50	$L_{2,500}$	$L_{4,500}$	$L_{6,500}$	$L_{8,500}$	$L_{10,500}$	$L_{12,500}$	$L_{14,500}$	$L_{16,500}$
750 ± 50	$L_{2,750}$	$L_{4,750}$	$L_{6,750}$	$L_{8,750}$	$L_{10,750}$	$L_{12,750}$	$L_{14,750}$	$L_{16,750}$
1000 ± 50	$L_{2,1000}$	$L_{4,1000}$	$L_{6,1000}$	$L_{8,1000}$	$L_{10,1000}$	$L_{12,1000}$	$L_{14,1000}$	$L_{16,1000}$

log mixed with traces of 4 adjacent different models and each model contributes 500 ± 50 traces to the mixed log.

Knowing the number and positions of change points in logs provides a gold standard to evaluate detection algorithms. A true positive (TP)/false positive (FP) detection is defined as a detection inside/outside a neighborhood of the precise concept drift timestamp. A false negative (FN) is defined as missing a detection within the neighborhood. The detection quality is measured by $F\text{-}score = 1/(1/recall + 1/precision)$ where $recall = TP/(TP + FN)$ and $precision = TP/(TP + FP)$. We refer to the radius of the neighborhood as error tolerance (addr. ET). For TP detections, the detection error is defined as the average of their distance to precise concept drift timestamp.

4.2 Experiments on MRID

First, we evaluated the impact of MRID by executing TPCDD with 4 different mixed logs ($L_{10,250}, L_{10,500}, L_{10,750}, L_{10,1000}$) under different MRIDs. Figure 8 presents the *F-score* with ET = 10.

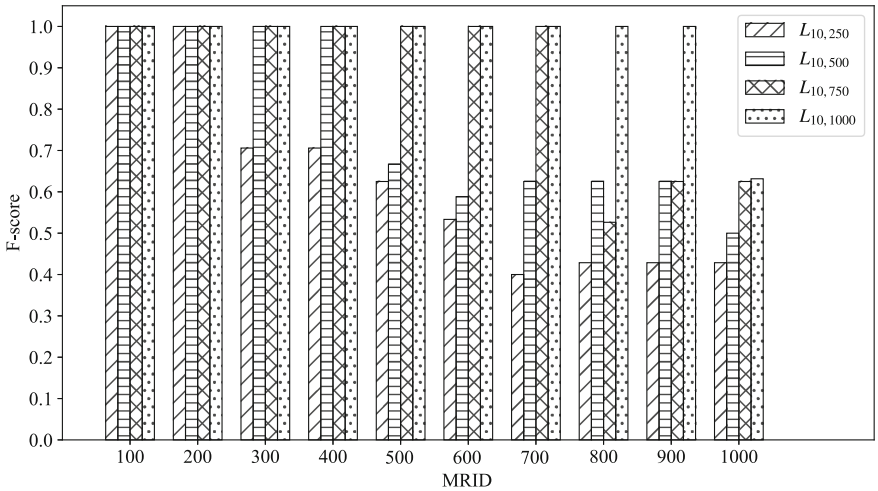


Fig. 8. F-scores under different MRIDs

As MRID grows, the *F-score* maintains steady at the beginning and then drops from a specific point. These points vary but lie around the corresponding log size of each model. This is because models of which the lifetime is less than MRID will be overlooked during detection. This experiment provides substantial evidence for the conclusion that TPCDD is not sensitive to parameter setting. As long as MRID is less than the log size of each model, the algorithm is able to fit in different log scale.

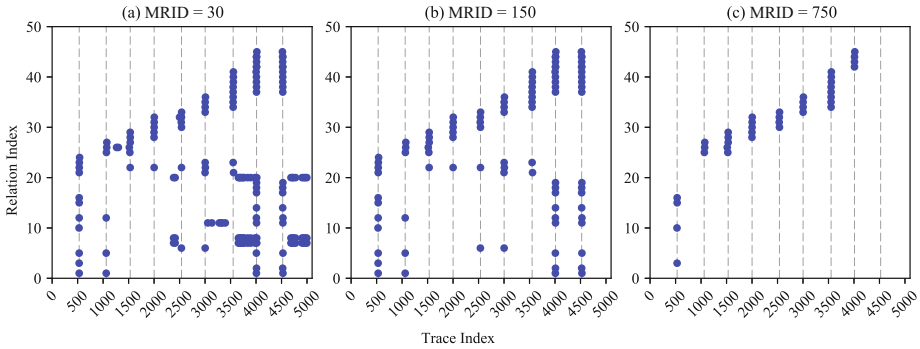


Fig. 9. Distribution of candidate change points under different MRIDs

Figure 9 provides us a visualization of the distribution of candidate change points (detected from $L_{10,1000}$). As is shown in the plot, most points lie around real change points. When MRID is too small (plot a), there exist some redundant points which are inside a model's lifetime. As MRID grows, the number of redundant points will gradually decreases. But in the main time, we may also lose some necessary points (plot c). This experiment confirms that our choice to use DBSCAN to combine candidate change points is reasonable and natural.

4.3 Experiments on *eps*

We also carried out experiments on the parameter of DBSCAN method *eps*. In this experiment, MRID was fixed to 100 and *eps* varied from 100 to 1100. Figure 10 shows the impact.

The F-score maintains steady at the beginning and drops from a specific *eps*. Take log $L_{10,500}$ as an example. When $eps \leq 400$, the F-score keeps high to 1.0. When $eps > 400$, the F-score falls off. This is because if *eps* is too large, a cluster would take in points in a wider range, resulting missing of real change points. Usually, *eps* is set to a small fraction (10% to 30%) of MRID.

4.4 Comparisons

According to Maaradji [12], their method outperforms the most mature approach [4] for process drift detection available. Since there was no algorithm better than

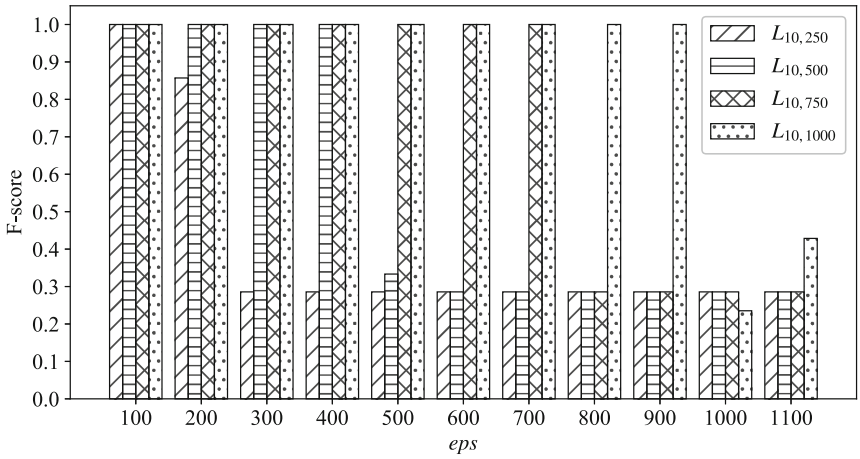


Fig. 10. F-scores under different *eps* settings

[12] at the time of writing, we compared TPCDD with [12] using synthetic logs listed in Table 3. As the adaptive method in [12] performs unstably, we chose to use only its fixed window mode here.

All 32 mixed logs were used in effectiveness experiment. We calculated the average for comparison. Figure 11 shows the result under different ETs. As is shown in Fig. 11, TPCDD performs better than Maaradji's, especially in situations where high precision (low error tolerance) is required. Maaradji's method cannot achieve a satifying F-score until ET increases to more than 100, which is unacceptable for small scale logs. Besides, the average detection error of TPCDD

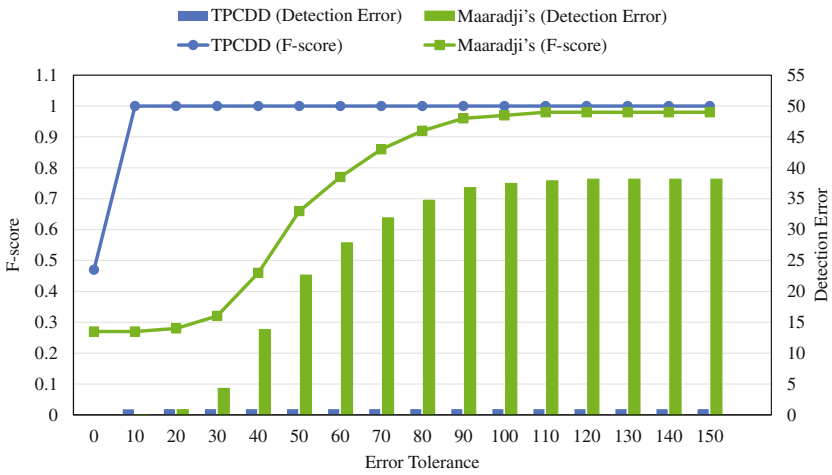


Fig. 11. F-score and detection error comparison between TPCDD and Maaradji's

is far less than Maaradji's, proving that the result of TPCDD is more close to real change points.

We also compared the time efficiency. TPCDD is implemented in Python while Maaradji's is implemented in Java. Figure 12 shows the time costs (not including file parsing time) over different mixed logs. Although Python is slower than Java in language perspective, TPCDD runs even faster than Maaradji's.

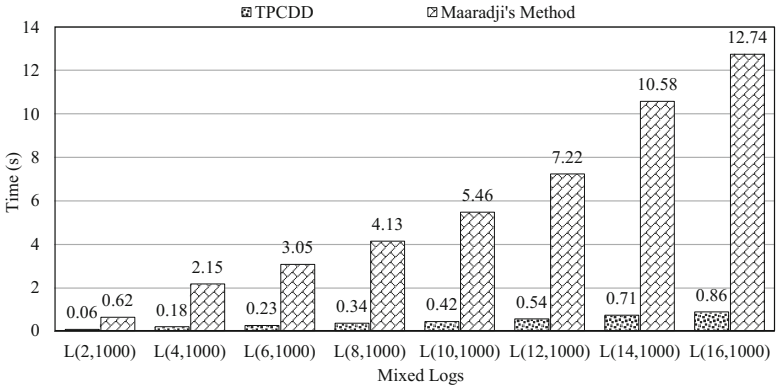


Fig. 12. Time efficiency comparison between TPCDD and Maaradji's

5 Related Works

A closely related problem to concept drift detection is change-point detection. The goal of the first is to detect changes that affect the mapping from the input space to the target concept, while the goal of the latter is to detect changes in the generating distributions of the time-series. Change-point detection is associated with homogeneity testing, in which, given two samples, one has to determine whether they were generated by the same distribution. Many attempts have been made to extend classical statistical tests for homogeneity for detection of changes in time-series [8,11]. Other methods are based on a predefined parametric model, such as the generating distribution [3,6,10], autoregressive models [17], and state-space models [7,14].

In the context of process mining, the input space refers to logs while the target concept is the underlying model. Concept drift detection is a new subfield in the process mining domain.

Carmona and Gavalda [5] first presented an online technique to detect concept drift by sequential monitoring of the logs. During each detection, an abstract interpretation in the form of polyhedron is first learned from the initial traces to represent the underlying concept. Then each subsequent trace is assessed whether it is within the polyhedron. Once the proportion of those traces inside the polyhedron drops exceed a specific threshold, a drift is detected. This method has a relation gap between polyhedron and traces. The ability of polyhedron to present

the underlying model lacks explanation. It is impractical due to its inability to pinpoint the exact moment of the drift and low efficiency.

Bose et al. [4] proposed a framework for analyzing concept drifts in process mining. They transform a trace stream into a numerical stream by extracting some features in the perspective of control flow. To detect change points, they consider a series of successive populations of values and investigate if there is a significant difference between two subsequent populations via statistical hypothesis testing. The significance probabilities of the hypothesis tests can be visualized as a drift plot to analyze drifts. This method is complicated and not automated. The feature sets in the paper expand the input to a large scale. It requires human participation in feature selection and change point recognition.

Based on [4], Martjushev et al. [13] extended the approach in three aspects: automating change points identification, using adaptive window strategy and proposing techniques for detecting gradual drifts and multi-order dynamics.

Maaradji et al. [12] proposed a method similar to [4] for detecting concept drifts in business process logs. Rather than defining lots of features, [12] introduced a structure called run to compress the original event log. A stream of traces can be transformed into a stream of runs. Its core idea is to perform statistical hypothesis testing over the distribution of runs in two consecutive time windows. The underpinning assumption is that if a change occurs at or around a given time point, the distribution of runs before and after this time point will be statistically different and the difference can be revealed by statistical hypothesis testing. By adaptively resizing the window, this method also strikes a trade-off between classification accuracy and drift detection delay. But in experiments we found that the adaptive window strategy sometimes performs much worse than fixed window strategy. Building upon this technique, Ostovar et al. characterized process drifts from event streams, in terms of the behavioral relations that are modified by the drift [15]. But it is not designed to detect change points.

The core idea behind [4,12,13] is measuring the differences between two consecutive sliding windows of traces. A small window size may result in redundancy while a large one may lead to miss of change points. It is difficult to achieve one size fits all. Besides, performing statistical hypothesis testing on the data stream transformed from trace stream is not an accurate way to measuring the difference.

6 Conclusion and Future Work

In this paper, we proposed a three-staged approach to detect concept drifts from event logs. An event log is transformed into a relation matrix by extracting direct succession and weak order relations of each trace. Rather than directly comparing two sublogs, we focus on specific relations and analyze their variation trend. We partition the life cycle of each relation by detecting *always* and *never* intervals (exceed a certain size) and obtain candidate change points. Finally, all the candidate change points are clustered and combined into real change points.

In the detection framework, one can easily enhance the recognition ability by customizing relations and add them into the relation matrix. From the result, we

not only learn when the changes occurred, but also distinguish successive models in the form of activity relation changes. Experiments showed that TPCDD is accurate and performs better than the state of the art in detecting sudden drift.

TPCDD currently focuses on offline scenarios. However, the framework is easy to fit in the online setting by doing little adjustment. Besides, situations where successive processes coexist for some time (denoted as gradual drift) is not supported in this paper and additional analysis for gradual drift is needed. Another avenue for future work is to make the method data and resource aware.

Acknowledgement. The work was supported by the National Key Research and Development Program of China (No. 2016YFB1001101) and the NSFC projects (No. 61472207, 61325008 and 71690231), and Tsinghua TNList Lab Key Projects.

References

1. van der Aalst, W.M.P., et al.: Process mining manifesto. In: Daniel, F., Barkaoui, K., Dustdar, S. (eds.) BPM 2011. LNBIP, vol. 99, pp. 169–194. Springer, Heidelberg (2012). doi:10.1007/978-3-642-28108-2_19
2. Accorsi, R., Stocker, T.: Discovering workflow changes with time-based trace clustering. In: Aberer, K., Damiani, E., Dillon, T. (eds.) SIMPDA 2011. LNBIP, vol. 116, pp. 154–168. Springer, Heidelberg (2012). doi:10.1007/978-3-642-34044-4_9
3. Basseville, M., Nikiforov, I.V., et al.: Detection of Abrupt Changes: Theory and Application, vol. 104. Prentice Hall, Englewood Cliffs (1993)
4. Jagadeesh, R.P., Bose, C., van der Aalst, W.M.P., Zliobaite, I., Pechenizkiy, M.: Dealing with concept drifts in process mining. IEEE Trans. Neural Netw. Learn. Syst. **25**(1), 154–171 (2014)
5. Carmona, J., Gavaldà, R.: Online techniques for dealing with concept drift in process mining. In: Hollmén, J., Klawonn, F., Tucker, A. (eds.) IDA 2012. LNCS, vol. 7619, pp. 90–102. Springer, Heidelberg (2012). doi:10.1007/978-3-642-34156-4_10
6. Gustafsson, F.: The marginalized likelihood ratio test for detecting abrupt changes. IEEE Trans. Autom. Control **41**(1), 66–78 (1996)
7. Kawahara, Y., Yairi, T., Machida, K.: Change-point detection in time-series data based on subspace identification. In: Seventh IEEE International Conference on Data Mining, ICDM 2007, pp. 559–564. IEEE (2007)
8. Kifer, D., Ben-David, S., Gehrke, J.: Detecting change in data streams. In: Proceedings of the Thirtieth International Conference on Very Large Data Bases, vol. 30, pp. 180–191. VLDB Endowment (2004)
9. Lamma, E., Mello, P., Riguzzi, F., Storari, S.: Applying inductive logic programming to process mining. In: Blockeel, H., Ramon, J., Shavlik, J., Tadepalli, P. (eds.) ILP 2007. LNCS, vol. 4894, pp. 132–146. Springer, Heidelberg (2008). doi:10.1007/978-3-540-78469-2_16
10. Lavielle, M., Teyssiere, G.: Detection of multiple change-points in multivariate time series. Lithuanian Math. J. **46**(3), 287–306 (2006)
11. Lung-Yut-Fong, A., Lévy-Leduc, C., Cappé, O.: Homogeneity and change-point detection tests for multivariate data using rank statistics. arXiv preprint arXiv:1107.1971 (2011)